Survival of the Cheapest: Cost-Aware Hardware Adaptation for Adversarial Robustness

Anonymous Authors

***Abstract*—Deploying adversarially robust machine learning systems requires continuous trade-offs between robustness, cost, and latency. We present an autonomic decision-support framework providing a quantitative foundation for adaptive hardware selection and hyper-parameter tuning in cloud-native deep learning. The framework applies accelerated failure time (AFT) models to quantify the effect of hardware choice, batch size, epochs, and test-set accuracy on model survival time. This framework can be naturally integrated into an autonomic control loop (monitor–analyse–plan–execute, MAPE-K), where system metrics such as cost, robustness, and latency are continuously evaluated and used to adapt model configurations and hardware selection. Experiments across three GPU architectures confirm the framework is both sound and cost-effective: the Nvidia L4 yields a 20% increase in adversarial survival time while costing 75% less than the V100, demonstrating that expensive hardware does not necessarily improve robustness. The analysis further reveals that model inference latency is a stronger predictor of adversarial robustness than training time or hardware generation.**

***Index Terms*—adversarial robustness, hardware-aware adaptation, survival analysis, self-adaptive systems**

I. Introduction

Cloud-native deployments of machine learning with deep neural networks are increasingly common in safety-critical classification tasks — with applications ranging from medical imaging [27] to aviation [44] and from security [5], [46], [65] to self-driving cars [11]. Statistical learning theory [59], [66] provides no guarantees about the generalisation performance of deep neural networks due to the massive number of tunable parameters. To overcome this, neural networks need large amounts of data [8], [24] to train ever-larger models [24], which has yielded increasingly marginal gains on test-set accuracy [62]. It is also clear that reaching safety-critical standards using test-set accuracy would require an infeasibly large test set [47].

Modern neural networks are massive — from AlexNet's 60M parameters [3] to Mamba's 8 billion [67], they now rank among the largest consumers of data-centre power [53]. At these scales, meeting even the weakest safety-critical threshold ($[10^{-12}, 10^{-15}]$ failure rate [1], [7], [19], [45]) would require billions of test samples per model change — computationally infeasible [47].

Furthermore, ensuring the robustness of ML models against adversarial noise has become a critical concern since inducing a failure at run-time has consistently been shown to be trivial [13]–[15], [17], [20], [50]. Collecting, labelling, and testing a new set of data for every software change — as required by law in most of the world [1], [7], [19], [45] — would be prohibitively expensive for these large models. Self-adaptive cloud-native systems face this challenge continuously: hardware configurations must be selected and updated at runtime, yet existing metrics provide no quantitative basis for such decisions under adversarial conditions. Therefore, a runtime-aware evaluation methodology — one that supports autonomous hardware selection under adversarial conditions — is required. In this work, we propose an autonomic decision-support framework based on accelerated failure-time (AFT) methods to predict model performance across hardware configurations and estimate deployment cost.

To tackle these problems, we present a methodology (Figure 1) and framework (Figure 2) that:

- Provides a quantitative decision-support component for self-adaptive systems, showing how AFT-derived cost and robustness estimates map naturally onto the monitor, analyse, and plan steps of a MAPE-K control loop [35].
- Demonstrates a scalable and effective method to train a model while simultaneously estimating the effect of various hyper-parameters, and
- Measures the power and monetary cost of deploying a model across different hardware architectures to model the trade-offs between deployment hardware and robustness.

This paper focuses on the Monitor, Analyse, and Plan phases of MAPE-K; live Execute-phase integration with a running autonomic system is addressed in future work (Section VIII).

II. Background

A. Autonomic Computing and MAPE-K

Autonomic computing refers to self-managing systems that adapt their behaviour in response to changing conditions without human intervention [35]. Kephart and Chess [35] define the MAPE-K reference model, which structures autonomic behaviour into four phases: *Monitor* (collect system metrics), *Analyse* (assess system state against goals), *Plan* (determine adaptation actions), and *Execute* (apply changes to the managed element), all sharing a common *Knowledge* base. In cloud-native ML deployments, this loop provides a principled structure for runtime hardware adaptation: cost and robustness metrics feed the monitor phase, while the analyse and plan phases select hardware configurations that maximise robustness per unit cost.

B. Cloud Architectures

A common approach is to use interconnected cloud-native services, where each service handles a specific task (*e.g.*

training, inference, preprocessing) [31], [54], [61], [72]. This architecture naturally maps onto the managed element in a MAPE-K loop, where the hardware configuration is the adaptation target.

In this work, we use Kubernetes [39] to orchestrate a multi-stage ML pipeline and evaluate its power usage and cost across different hardware architectures.

C. ML Pipelines

ML pipelines are complex, long-running workflows with many tunable hyper-parameters, requiring careful management [26], [39], [71]. Data is typically split into training and test sets, with the latter used to evaluate model configurations and generalisation.

Hardware differences (*e.g.* VRAM) influence optimal configurations, especially when considering robustness and cost. Training and inference may also run on different hardware to optimise efficiency, as some devices are specialised for training (*e.g.* V100, P100) and others for inference (*e.g.* L4).

D. Classifiers

We consider ML classifiers $K(x;\theta)$ with parameters θ, true labels y, and predictions $\hat{y} = K(x;\theta)$ for a mini-batch x of size N. The loss function $L(y,\hat{y})$ measures prediction error.

Due to model complexity, training typically uses stochastic gradient descent (SGD), which updates parameters using mini-batch gradients over multiple epochs:

$$\theta^{(i+1)} = \theta^{(i)} - \eta \nabla_{\theta^{(i)}} L(y, K(x, \theta^{(i)})), \tag{1}$$

where η is the learning rate and gradients are approximated using batches of size b.

E. Learning Rate Selection

The learning rate strongly affects both performance and training cost. Smaller values improve accuracy but slow convergence, while the optimal scale depends on factors such as batch size, data dimensionality, and training duration. Larger GPU memory enables larger batches, reducing updates per epoch.

An effective learning rate balances fast convergence and accuracy, but in cloud environments, hardware variability requires empirical tuning. Since compute is billed, optimising training time is also cost-critical. To jointly optimise training efficiency and robustness (λ_{ben}, λ_{adv}), we use TPE optimisation (Section III-C).

F. Adversarial Attacks

An adversarial attack is a deliberate attempt to manipulate an ML model by introducing carefully crafted input perturbations that cause incorrect predictions or otherwise undesired outputs. In this work, we focus on *evasion attacks*, where the attacker perturbs the input at inference time to induce misclassification. Formally, *adversarial success* or *accelerated failure* is one in which

$$\hat{y} \neq \hat{y}_a = K(x + \varepsilon; \theta), \tag{2}$$

where ε is a noise distance bounded by $0 < \varepsilon \leq \varepsilon^*$. Additionally, one can measure the accuracy of the model when tasked with these adversarial samples, giving us a metric called *adversarial accuracy*, which is Eq. 4 calculated on the perturbed samples. These attacks can occur during various stages of the ML pipeline, including during training [10], [55], inference [16], [51], or deployment [13], [14], [16]–[18], [37], [42]. One of many possible attacks is designed to induce failures as quickly as possible, and is conveniently called the *fast gradient method* [28]. It works by applying noise to a set of samples, x, to generate adversarial examples, x_a, such that,

$$x_a = x + \eta \cdot \text{sign}(\nabla_x L(y, K(x, \theta))). \tag{3}$$

G. Adversarial Analysis

In safety- and security-critical domains, a system is considered broken if its failure rate exceeds the applicable standard [19], [56]. Meeting even the weakest IEC 61508 threshold requires billions of test samples per model change — computationally infeasible for continuously-updated ML systems. Adversarial failure analysis [9], [14], [47] circumvents this by deriving precision from small sample sets. In self-adaptive systems, the Monitor phase must track this degradation at runtime efficiently.

III. Survival Analysis for Cost-Aware Robustness Estimation

AFT models are statistical models used to analyse multivariate effects on the observed failure rate to predict the time-to-failure across a wide variety of circumstances [12], [36], mapping the relationship between model tuning parameters and performance to provide an analytical foundation for runtime hardware selection in self-adaptive systems.

A. Accuracy

Accuracy measures the vulnerability or susceptibility of the model to failures. A larger accuracy indicates a higher rate of true classifications; while this reflects better generalisation on clean data, high benign accuracy does not imply adversarial robustness, as the results in Section VI confirm. Throughout, we use the terminology *benign* accuracy to refer to the performance on the test set using unperturbed data and *adversarial* to refer to the performance in the presence of additive noise that is intended to confuse the model. The subscripts *ben* and *adv* are used respectively. The accuracy, λ, is defined as

$$\lambda := \text{Accuracy} := 1 - \frac{\text{False Classifications}}{\text{Total Classifications}}, \tag{4}$$

which is generally assumed to indicate the rate of successes in real-world data sampled from the same distribution as the training data [64]. However, it ignores run-time cost [8], [24] and adversarial noise [20], and adversarial counter-examples consistently expose this split as optimistic [10], [13], [14], [37], [47].

B. Failure Rate

The failure rate refers to the percentage or proportion of examples that cause the targeted ML model to misclassify or produce incorrect outputs [47]. To encompass the cost of a particular model or attack, the proposed methodology considers failures to be a function over some time interval (*e.g.* training time, inference time, attack generation time, *etc.*) and some covariates, θ, so that the failure rate is the average time until a failure in a time interval around time, t, such that:

$$h_\theta(t) := \frac{p(\text{False Classification} \mid \theta)}{\Delta t} t,$$

where $p(\text{False Classification} \mid \theta)$ is the probability of a false classification given a particular set of hyper-parameters, θ, Δt is a time interval, and t is a point in time. Note that $1-\lambda$ is an estimate of this value when $\Delta t = t$ and converges to this value as the number of samples, $N \to \infty$. By modelling accuracy as a function of time, one can then compare the probability of failure to the cost, which is also measured in time, allowing one to make deployment decisions based on the risk analysis standards outlined in IEC61508 [19].

C. Optimisation

ML models are typically trained by examining the effect of the entire hyper-parameter space on the resulting accuracy. However, the number of hyper-parameter combinations is often infinite or at least exponential in the number of hyper-parameters, making it infeasible to exhaustively evaluate the entire hyper-parameter search space. Additionally, the goals of test-set accuracy and adversarial accuracy are often at odds — with several researchers noting an inverse relationship between model accuracy and model robustness [14], [25], [47]. Therefore, a proper search would keep this dual-objective in mind. In addition, we attempt to minimise the training time for each piece of hardware since the optimal batch size and, therefore, the learning rate will be determined by the VRAM, the bit depth of the data as well as the size and bit depth of the model. To maximise adversarial and benign accuracy simultaneously while minimising training time, we propose the use of the Tree-Parzen Estimator because it has been shown to converge over hundreds of trials rather than the thousands of trials typical of other multi-objective optimisation algorithms like CMAES or NSGA-II [2], [52], [68]. Here, λ_{ben} denotes benign accuracy (Eq. 4 evaluated on unperturbed samples) and λ_{adv} denotes adversarial accuracy (Eq. 4 evaluated on adversarially perturbed samples).

D. AFT Models

AFT models are widely used in industrial, medical, and risk-mitigation contexts [12], [36] to model covariate effects on expected time-to-failure; here we apply them to ML by inducing adversarial failures to measure generalisation performance under varying hardware configurations.

We model the *survival time*, $S_\theta(t)$, as a function of time, t, and some set of model parameters, θ, such that,

$$S_\theta(t) = p(T > t \mid \theta) = \exp\left(-\int_0^t h_\theta(u)\, du\right)$$

where $p(T > t)$ is the probability that a model has not failed by time t ("survives" beyond time t). The expected survival time is

$$\mathbb{E}_{S_\theta}[T] = \int_0^{t^*} S_\theta(u)\, du, \tag{5}$$

where t^* is the latest time observed in the survival data. However, modelling $S_\theta(t)$ requires a choice of modelling function for S_θ. The Log-Logistic, Log-Normal, and Weibull functions are widely used alternatives [36], [48]. For each trial, one can measure the attack generation time to define the time interval and the accuracy to estimate the number of failures and successes in that time interval.

1) Survival Time: By using adversarial samples (see Equation 2), failures can be induced. The likelihood of those failures depends on the amount of adversarial noise, ε, since adversarial noise is known to *induce* failures. In the language of accelerated failure time models, this can be expressed in terms of the accelerated failure time assumption [36]

$$S_\theta(t) = S_0\left(\frac{t}{\phi_\theta(x)}\right), \tag{6}$$

where ϕ_θ is the acceleration factor, described by the joint effect of the covariates, such that

$$\phi_\theta(x) = \exp\left(\theta_0 x_0 + \theta_1 x_1 + \cdots + \theta_n x_n\right),$$

and where $x = (x_0, \ldots, x_n)$ is a vector of covariates and $\theta = (\theta_0, \ldots, \theta_n)$ describe the fitted parameters, and ε is used as one of the covariates. That is, when the adversarial noise level changes, it will also alter the expected survival time of a sample. This assumption means we can evaluate the generalisation performance using a small number of adversarial samples with our precision coming from careful time measurements rather than massive test sets [36], [48].

E. Choosing the best AFT Model

To choose a best-fit from the possible AFT functions, one should prepare the collected metrics and scores and then compare them using *e.g.* Akaike Information Criterion (AIC), Bayesian Information Criterion (BIC), or Concordance, as per the best practices for this methodology [12], [36]. For AIC and BIC, that means choosing the smallest value. Concordance, however, is a number between 0 and 1 that quantifies the degree to which the survival time is explained by the model, where a 1 reflects a perfect explanation [36] and 0.5 reflects random chance. By evaluating $\mathbb{E}_{S_\theta}[T]$ under extreme perturbations, one can test the model and minimise the number of evaluated samples [12], [36] rather than relying on the $> 10^{12}$ samples as required by IEC61508 [19]. While the aforementioned metrics measure goodness-of-fit, one should conduct a *survival probability calibration*, where one fits the AFT model to the data and compares it to a cubic-spline that is meant to capture the un-modelled relationship between failures and time [6]. These plots can be used to visually inspect the goodness-of-fit of various models, as in Figure 6, and this method is called *survival probability calibration*. The integrated calibration index (ICI) as well as

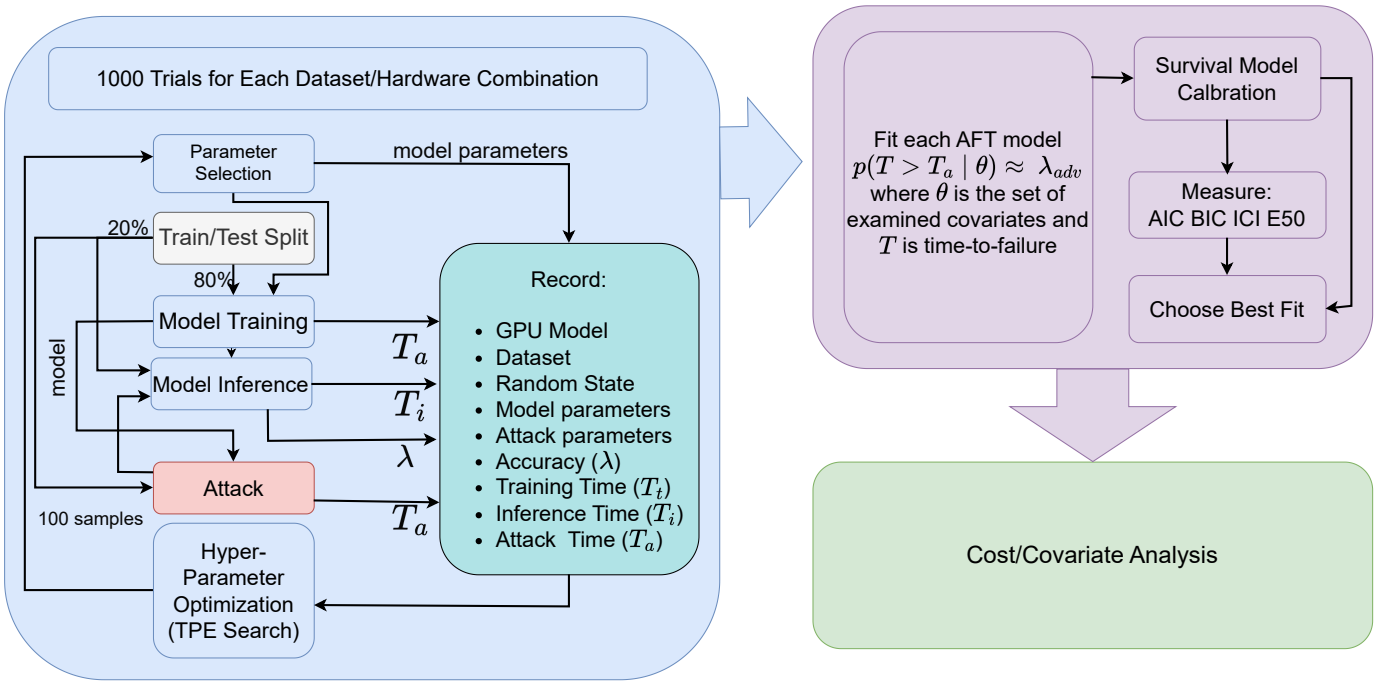

Fig. 1. For each dataset and hardware combination, a state parameter was chosen at random to decide the test and train sets. Next, model parameters and attack parameters are chosen at random. After 128 random trials, the TPE algorithm attempts to maximise benign and adversarial accuracy while minimising training time by tuning the model parameters. The random seed for the data split and the attack parameters are sampled independently from this optimisation, which is why they are coloured differently. The model tuning (blue-box) is discussed in Section III-C. After the trials are completed, several AFT models are fit (see Section III-D) and compared (see Section III-E) using the process depicted in the purple box. Finally we conduct the cost analysis outlined in Section IV (green box).

the error at the 50th percentile E50 [6] can then be calculated. These are the mean absolute difference between observed and predicted probabilities and the median absolute difference between observed and predicted probabilities, respectively [6]. Additionally, the data were split into a training set that was used to fit the model (80%) and an unseen test set (20%), and the concordance, ICI, and E50 were measured for both. The resulting expected survival time estimates serve as the analytical input to the Analyse and Plan phases of a MAPE-K loop, enabling hardware configurations to be ranked by cost-normalised robustness.

IV. Cost and Robustness Metrics for Hardware Adaptation

Building on the survival analysis in Section III-D, this section defines the cost and robustness metrics that feed the Plan phase of a MAPE-K loop, enabling runtime hardware selection decisions. Cost is considered across three dimensions: computational efficiency (TRASH score), monetary deployment cost, and energy consumption. Together these metrics quantify the marginal risk [19] and deployment cost of a given hardware configuration: the benign and adversarial accuracies (see Equations 2 and 4), the model training time, t_t, the model inference time or latency, t_i, the attack generation time, t_a, the cost per hour for a particular hardware, C, as well as the power consumption, P, of each tested model and attack.

A. Accuracy

Under the AFT framework (Section III-A), adversarial accuracy serves as a measure of survival time across a specified time period, as outlined in Figure 1.

B. Training Time

The training time, T_t, is the time it takes to evaluate n samples, where t_t is the training time per sample. It is defined as

$$T_t := t_t \cdot n \cdot m,$$

where m is the number of epochs.

C. Latency

Latency is the time it takes to respond to a query. We assume that latency per sample is

$$T_i := t_i \cdot n,$$

which will be driven by the memory bandwidth (measured in bits/second) of a given CPU or GPU and the size [60] and complexity [32] of a given neural network architecture.

D. Attack Generation Time

A successful attack is one that induces failure in a model. That is, the expected survival time, $\mathbb{E}_{S_\theta}[T]$, can be thought of as the average time it takes for an attacker to induce a change

in the model output. Assuming a uniform distribution, T_a, for n samples, i iterations, and attack time per sample, t_a, as

$$T_a := t_a \cdot n. \tag{7}$$

In reality, the attack time will not be drawn from a uniform distribution and prior research [48] has shown that by treating model and attack parameters as covariates, one can model the survival time accurately by using an AFT model, such that:

$$t'_a \approx \mathbb{E}_{S_\theta}[T] = \int_0^{t^*} S_\theta(t)dt. \tag{8}$$

E. TRASH Score

With an estimate of the expected survival time in hand, we quantify the cost-normalised failure rate, or the ratio of training time to attack time. Assuming that the cost scales linearly — as discussed above — the model builder's cost is proportional to the training time ($C_t \propto t_t$), and the attacker's cost is proportional to the attack time, which is approximately the expected survival time ($C_a \propto t'_a \approx \mathbb{E}_{S_\theta}[T]$). Using the definition of ε from Equation 2 and definition of t'_a from Equation 8, we can express the cost of failure in adversarial terms as follows:

$$\text{TRASH} = \frac{t_t}{\mathbb{E}_{S_\theta}[T]} = \frac{t_t}{t'_a}. \tag{9}$$

F. Monetary Deployment Cost

Furthermore, we approach the cost of deployment at two scales. Firstly, we consider the cloud-rental scale, where a small business might test and deploy a model using *e.g.* the Google Cloud Platform (GCP) compute costs as a measure of total cost. However, at a certain scale or with certain applications, it is more appropriate to talk about cost in terms of power (*e.g.* to deploy a self-driving car with a useful operating range). Finally, we define metrics that provide an efficient way to minimise the latency and cost of deployment, and to maximise the generalised performance of a model. We define the training cost as

$$C_t = C_h \cdot T_t,$$

where C_h is the cost per unit time for the hardware, T_t is the training time, T_i is the inference time, and T_a is the attack generation time, as defined above. Inference and attack costs follow analogously, denoted with i and a subscripts respectively.

G. Power

The power consumption for a particular piece of hardware, P_h, measured in Watts (Joules per second), can be thought of similarly such that the total power consumption of model training, inference, and attack is denoted with t, i, a subscripts respectively. In this work, power is monitored in real time using KEPLER [4], providing the energy cost metrics that feed the Monitor phase of the MAPE-K loop.

Together, the TRASH score, monetary cost, and power consumption form the decision criteria for the Plan phase of a MAPE-K loop, enabling a self-adaptive system to select the hardware configuration that maximises robustness per unit cost.

V. Experiments

This section details the implementation of the methodology in Sections III–IV, with each GPU configuration evaluated as a candidate Execute-phase adaptation target.

A. Cloud Platform and Hardware

To conduct the experiments and have access to different types of hardware, we utilised GCP. Six virtual machines running Container-Optimised OS provided by GCP constituted the test-bed. Using Google Kubernetes Engine 1.27.3 and Containerd 1.7.0, a cluster consisting of six worker nodes was created. Three worker nodes were responsible for running the monitoring platforms — Prometheus 2.47.2 and Grafana 10.2.0. These nodes were of the "e2-medium" instance type provided by GCP. In total, three GPU architectures were used — the Nvidia P100, V100, and L4. For P100 and V100 GPUs, the "n1-standard-2" type was used for the nodes and for L4 GPUs the "g2-standard-4" was used.

To assess the energy consumption of the experiments, KEPLER was deployed on each node to measure the power consumption of each experiment [4]. KEPLER collects per-pod energy metrics within the Kubernetes cluster and exports them as Prometheus metrics [58]. A diagram of the cloud architecture can be found in Figure 2. Finally, all of the experiments were restricted to a $1,000 budget (a research grant from Google). Approximately 10% was used for development and 90% for the evaluations. Over the course of the evaluations, 6% of the budget went to storage, 6% to monitoring, and the remaining 88% went to the GPUs.

B. AFT Models

For each hardware configuration and dataset, the TPE algorithm was used to select the hyper-parameters [52], [73], and it was iterated for 1,000 trials (including 128 random startup trials used to initialise the TPE surrogate model). We used three optimisation criteria: benign accuracy, training time, and adversarial accuracy, seeking to maximise both benign and adversarial accuracy while minimising training time (and therefore deployment cost). We selected a set of parameters as per the TPE algorithm and trained on 80% of the samples for each of the MNIST, CIFAR10, and CIFAR100 datasets. Of the remaining samples, 100 were withheld to be attacked and used to evaluate the adversarial accuracy. Figure 1 illustrates this methodology. For each dataset, we tested this on 10 random splits of the data to create 10 unique test/train pairs. For each trial, we recorded attack generation time, model training time, model inference time, benign accuracy and adversarial accuracy, and the size of the training set, test set, and attack set. Using these values, we fit an AFT model to the number of failures (indicated by accuracy and sample size) and the attack generation time after adding dummy variables for the dataset and hardware device. Additionally, we approximated the AFT model using the methodology depicted in Figure 1.

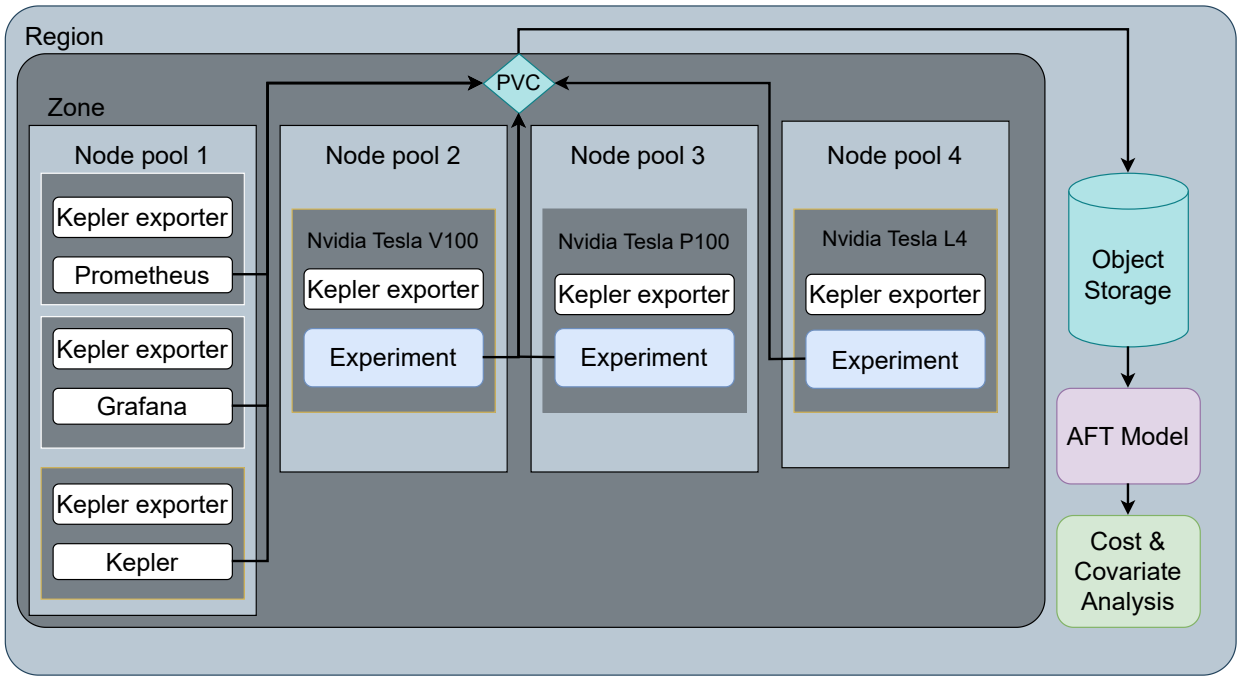

Fig. 2. For our experiments we used four node pools from Google Cloud Platform, each with a particular responsibility. The first node pool included three nodes responsible for hosting monitoring services such as Prometheus and Grafana. The other node pools each had one node with a specific GPU. The KEPLER exporter was deployed on each node as a DaemonSet to monitor resource usage. All experimental and monitoring data was stored using persistent volume claims (PVCs), which provide Kubernetes-managed persistent storage. The blue experiment, blue-green persistent storage, green analysis component, and purple AFT component correspond to the same colors in Figure 1.

C. Datasets

The AFT models were evaluated using the MNIST [23], CIFAR10 [38], and CIFAR100 [38] datasets, chosen primarily for their standardised use in adversarial analysis [14], [20], [43], [49] and decades of experimental results. Before training, we centered and scaled the data so that the attack distance would be comparable for all tested datasets. Furthermore, to reduce the complexities of system overhead, distributed or federated training, and the effect of shared cloud environments, we restricted ourselves to datasets that were small enough to reside entirely within GPU memory with the model, since the disk read speed in cloud environments is highly variable.

D. Models

The evaluations were restricted to a single model. Primarily, this was done to meet the budgetary constraints since evaluating more models would mean evaluating fewer pieces of hardware. As discussed in Section II-E, the relationship between hardware specifications, hyper-parameters, and performance is highly complex and hard to predict. So, we sampled learning rates $\in [10^{-6}, 1]$, batch sizes $\in [1, 10^5]$, and epochs $\in [1, 50]$ for MNIST and CIFAR10 on the P100 and V100. For CIFAR100, the range of the tested epochs was increased to be $\in [1, 100]$. The Feature Squeezing defence [70] was used to evaluate the efficacy of different bit-depths on the L4 hardware, as provided by IBM's adversarial robustness toolbox [50] with bit depths $\in [4, 8, 16, 32, 64]$ which casts the inputs into `pytorch`-compatible arrays. Model parameters were chosen using the `optuna` optimisation framework, the configuration was handled by `hydra`, and `dvc` was used to ensure reproducibility and aid in collaborative development.

E. Attacks

To examine the effect of model parameters at run-time, evasion attacks, which attack the model at the prediction stage, were examined. Prior research [47], [48] has shown that the Fast Gradient method (see Eq. 3) is consistently the most effective at inducing a large number of failures in a small amount of time. To evaluate the effect of adversarial noise on the samples, the noise levels were varied $0 < \varepsilon \leq 1$. This was done using the `adversarial-robustness-toolbox` package maintained by a team at IBM [50].

F. GPU Configurations

Several hardware configurations were tested that had various hourly costs, peak power demands, and theoretical memory bandwidths. The V100 was chosen as a baseline, since it is routinely used in the literature [63], [69]. The P100 architecture comes from the same line of server-grade GPUs, but from an older generation. The L4, however, is advertised as a machine built for inference — not training — relying on a smaller number of bits per tensor core. Consequently, the number of operations per second depends on the bit depth of the data and model weights, with peak numbers outlined in Table I for 8-bit inputs. The rental cost of the hardware, measured in United States Dollars per hour, indicates the

operating cost of a given model. To calculate this, the price per hour from each cloud service pricing page [29], [30] was used, and the cost of training (C_t) and the cost of inference (C_i) were calculated from the cost of hardware (C_h), the training time (T_t), and the inference time (T_i).

G. Survival Analysis

In addition to the optimisation criteria of benign/adversarial accuracy and training time, prediction times, attack times, power consumption, batch size, and number of epochs were also collected to be used as covariates in the AFT model, $S_\theta(t)$. To fit the AFT model and to plot the effect of the covariates, the `lifelines` Python package was used [21]. The Weibull, Log Logistic, and Log Normal AFT models (see Section III) were also compared.

VI. Results and Discussion

A. Accuracy

Figure 3 shows the benign (left) and adversarial (right) accuracies for all datasets and hardware. It demonstrates little to no change in accuracy or adversarial accuracy, regardless of hardware. The benign accuracy decreases with difficulty (CIFAR10 vs MNIST) or with the number of classes (CIFAR10 vs CIFAR100). For all three datasets, the adversarial accuracy becomes the reciprocal of the number of classes (*i.e.*, the accuracy we would expect with random data), demonstrating the efficacy of the attack outlined in Section II-F. These accuracy metrics form the Monitor-phase inputs that feed the Analyse phase of a MAPE-K loop evaluating hardware robustness.

B. Time, Power, and Monetary Cost

The power consumption during training, inference, and attack generation for all hardware and datasets is illustrated in Figure 4. It tracks monetary cost (Figure 5) closely, likely because power is the predominant operating cost for data centres [22], so it is unsurprising that cloud billing is correlated with the power requirement. Furthermore, we see that the largest dataset (CIFAR100) and smallest GPU (L4) require the least amount of power (Figure 4) for prediction, while differences between hardware and datasets are insignificant for the training and attack metrics.

The monetary cost for each dataset and each piece of hardware is shown in Figure 5. For all three datasets across all three pieces of hardware, the cost of training on a single sample often exceeds the cost of attacking a single sample. In the best-case scenario, they are comparable, but attacks consistently succeed with only 100 samples (Figure 3) while model training requires orders of magnitude more. Together, the power and cost measurements in Figures 4–5 constitute Monitor-phase observables that a MAPE-K Analyse phase can use to rank hardware configurations by cost-effectiveness.

Figures 4 and 5 show the distributions of power consumption and monetary cost during training, inference, and attack generation across all hardware configurations and datasets.

C. AFT Models

Table II shows the performance metrics for all three AFT models, as outlined in Section III-D. The concordance is both strong (> 0.5) and similar for all three AFT models and both the train and test sets. We observed no more than 1% mean absolute error in the probabilities ICI and no error in the median probability E50 across all three AFT models. Figure 7 shows the log-scale coefficients for the AFT model. It shows that epochs, batch size, and training time have no effect on the survival time. However, we can clearly see that inference time is as strong an indicator as the attack noise distance, revealing that model speed is nearly as important as the noise value of the attack (ε). Furthermore, we see that benign accuracy is negatively correlated with the survival time, confirming previous assertions that robustness ($S_\theta(t)$) is inversely related to benign accuracy (which is the standard indicator of generalisation performance) [14].

Figure 7 further shows that hardware choice has a relatively small effect on robustness — the L4 increases adversarial survival time by approximately 20% and the P100 decreases it, both relative to the V100 — despite the much larger disparity in cost outlined in Table I.

D. Why Cost Matters

In security analysis, it is routine to think in optimistic terms for both the attacker and defender. In cryptography, these ideal attack- and defence-scenarios are used to test the computational feasibility of subverting a particular cryptographic system [34], [41] (*e.g.* whether or not a given cryptographic method should be considered "broken"). By using a whitebox attack to generate the failures, we ensure that our defender operates under the worst-case scenario while the attacker operates under their best-case scenario to yield a worst-case failure rate. As such, we centre our analysis on the whitebox Fast Gradient Sign Method (FGSM) [28] (see Eq. 3), which is both effective and fast [47]. If the cost to a model builder is much larger than the cost to an attacker, then it is clear that the model is "broken" in this cryptographic sense and can be discarded as ineffective [48]. Figure 8 depicts the TRASH score defined in Equation 9, which quantifies the per-sample ratio of training to attack time. Even with the reduced GPU bandwidth (Table I), the L4 model is superior in terms of both cost (Table I) and robustness (Figure 7), presumably since the extra bit-depth provided by the P100 and V100 ends up being useless noise on 8-bit images like MNIST and CIFAR. This finding directly informs the Plan phase of a MAPE-K loop: a self-adaptive system can confidently select the least expensive hardware configuration without sacrificing adversarial robustness.

E. Advantages of this Methodology

The primary advantage of this methodology over the traditional test/train split measurements is that the precision of the latter is determined by the number of samples, whereas the precision of the survival time estimate is driven by the resolution of our timing measurements. For test-set accuracy,

	V100	P100	L4
Cost (USD/hour)	2.55	1.60	0.81
Power (Watts)	250	250	72
Memory Bandwidth (GB/s)	900	732	300

TABLE I

HARDWARE SPECIFICATIONS FOR THE TESTED GPUS. THE SPECIFICATIONS WERE RETRIEVED FROM NVIDIA'S WEBSITE AT THE FOLLOWING LINKS: V100 DATASHEET, P100 DATASHEET, AND L4 DATASHEET. PRICES WERE RETRIEVED FROM GOOGLE CLOUD PLATFORM FOR THE `EUROPE-WEST4` REGION ON 3 DECEMBER 2023.

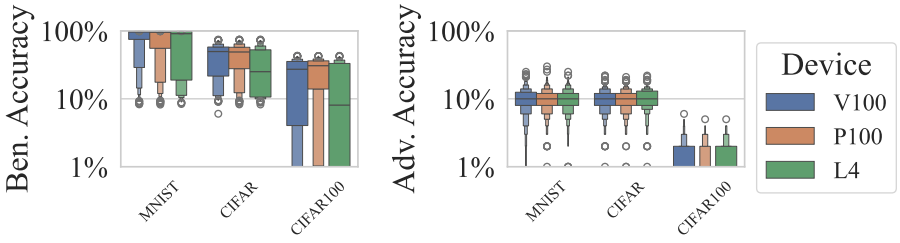

Fig. 3. Benign and adversarial accuracy across all hardware and datasets for all 1000 trials using plots that depict the distribution of the y-axis values using the width of the plot. Each colour is a different device and the datasets are displayed along the x-axis. Outliers are denoted with a white dot.

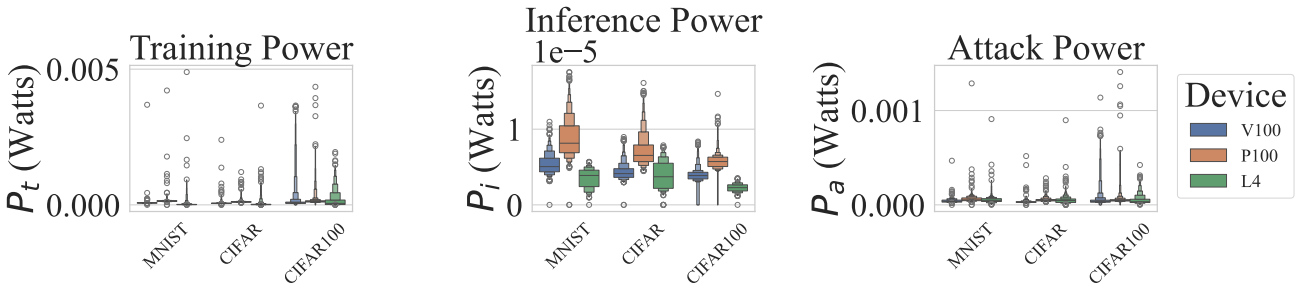

Fig. 4. This depicts the power consumption during training, inference, and attack generation for all hardware and datasets for all 1000 trials using plots that depict the distribution of the second axis values using the width of the plot. The power per sample was assumed to be uniform across the batch of samples for each training, inference, or attack measurement. Each colour is a different device and the datasets are displayed along the first axis. Outliers are denoted with a white dot. For these plots, the second axes have been scaled by the number of samples for the sake of comparison.

meeting the weakest safety-critical IEC standard (one failure in a million) would require many millions of samples to confidently conclude that a model is safe. However, the presented AFT model has an average error rate of 1% while requiring a very small number of samples (10 sets of 100) (see Table II). Because AFT models require far fewer samples than in-distribution test sets, they can serve as lightweight unit tests for ML components — quantifying the marginal risk per IEC 61508 [19] without requiring full-system integration or live deployments [40], [57]. Additionally, since AFT models have strong predictive power for untested hyper-parameter combinations, this method has the potential to reduce the search space by quickly eliminating candidates that are unlikely to increase the survival time. Furthermore, the resulting cost and robustness estimates serve as the analytical foundation for a MAPE-K control loop [35], positioning the methodology as a quantitative decision-support component for self-adaptive cloud-native ML deployments.

VII. SCOPE AND LIMITATIONS

While this work focuses on adversarial robustness, the cost and survival analysis framework presented here is general and could be applied to any quantifiable failure condition in ML systems — such as hardware faults, distribution shift, or data corruption.

We have taken much care to conduct all timing measurements as carefully as possible. Primarily, to minimise timing jitter and account for GPU parallelisation, we assumed that the time-to-failure during the benign and adversarial accuracy measurements was uniform across the samples in each trial. While it is very possible (if not altogether guaranteed) that some classes or samples are easier to attack than others, we assume that this averages out over the 100 samples given to each attack. Further work examining the effect of imbalanced datasets on survival time distributions is outside

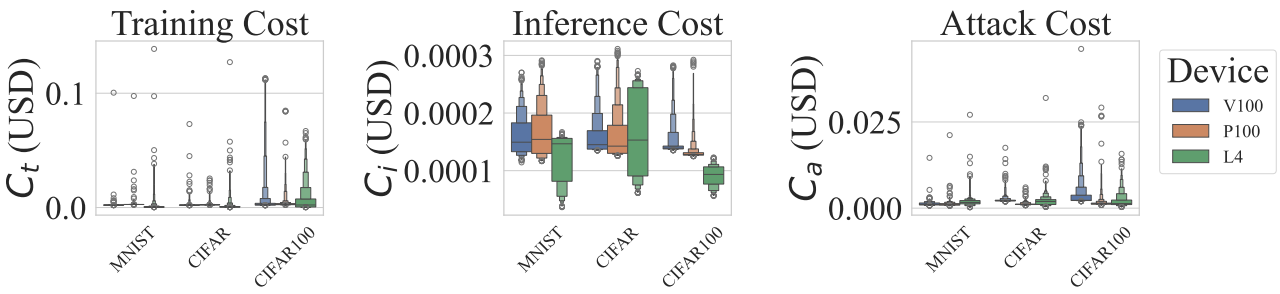

Fig. 5. This depicts the monetary cost during training, inference, and attack generation for all hardware and datasets for all 1000 trials using plots that depict the distribution of the second axis values using the width of the plot. The cost per sample was assumed to be uniform across the batch of samples for each training, inference, or attack measurement. Each colour is a different device and the datasets are displayed along the first axis. Outliers are denoted with a white dot. For these plots, the second axes have been scaled by the number of samples for the sake of comparison.

TABLE II
COMPARISON OF AFT MODELS ACROSS ALL HARDWARE AND DATASETS. AIC AND BIC ARE MEASURES OF GOODNESS-OF-FIT, WITH A SMALLER VALUE BEING PREFERRED. CONCORDANCE (CONC) IS A VALUE BETWEEN 0 AND 1 THAT REFLECTS WHAT PROPORTION OF EVENTS (FAILURES) CAN BE EXPLAINED BY THE MODEL. ICI MEASURES THE AVERAGE ERROR BETWEEN A CUBIC-SPLINE AND THE MODEL AND E50 MEASURES THE MEDIAN ERROR BETWEEN THE SPLINE AND THE MODEL. THESE ARE MEASURED ON BOTH THE TRAIN AND TEST SETS OF THE COLLECTED DATA, WITH THE LATTER BEING DENOTED "TEST".

	AIC	BIC	Conc	Test Conc	ICI	Test ICI	E50	Test E50
Weibull	$-2.11 \cdot 10^4$	$-2.11 \cdot 10^4$	0.84	0.83	0.00	0.01	0.00	0.00
Log Logistic	$-2.18 \cdot 10^4$	$-2.18 \cdot 10^4$	0.84	0.83	0.01	0.01	0.00	0.00
Log Normal	$-2.25 \cdot 10^4$	$-2.25 \cdot 10^4$	0.84	0.83	0.01	0.00	0.00	0.00

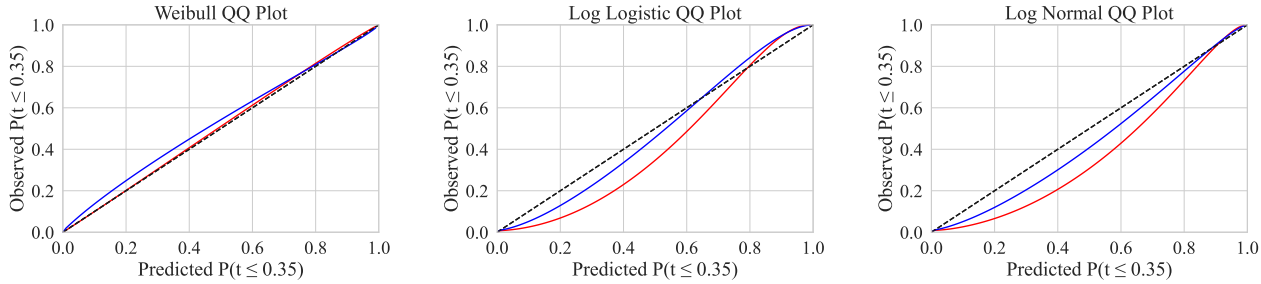

Fig. 6. The red and blue lines depict the relationship between the modelled number of failures (x-axis) and the measured number of failures (y-axis) for the training set (blue) and test set (red). The dotted line indicates a cubic spline fit to $S_\theta(t = t_a) \approx \lambda_{adv}$. A model that fits this cubic spline exactly would be a diagonal line at $y = x$. The process of comparing the fitted AFT model to a default cubic spline is called *survival probability calibration* and is discussed in more detail in Section III-E.

the scope, though the methodology would remain identical. However, given the strong results and uniformity across AFT functions in Table II, the uniform time assumption appears to be insignificant. However, accounting for the failure rate per sample or class is likely to account for some of the remaining unexplained variance. Additionally, we chose a model and set of datasets small enough to fit entirely in GPU memory to minimise the confounding factors around parallelised, distributed, and/or federated learning as well as the complexities around storage hardware, file systems, and various access mechanisms. Larger models acting on larger datasets are likely to perform better on hardware with a higher GPU bandwidth, but the 24GB of VRAM provided by the L4 should be more than sufficient for vision tasks based on 8-bit images standard in the literature (*e.g.* MNIST, CIFAR10, CIFAR100); indeed, at the time of writing, 98.9% of publicly available datasets on Kaggle are smaller than 24GB [33]. Furthermore, the attack batch size and the training batch size were set to be the same for every trial for each dataset and piece of hardware so that the attack timing would mimic the usage of regular users. If anything, the smaller sample size of the attacks means that proportionally more parallelisation overhead is needed per sample, leading to an underestimate of the attack efficacy compared to the benign measurements. In general, distributed/federated models were out of scope, but could be evaluated using the same cost and survival analysis

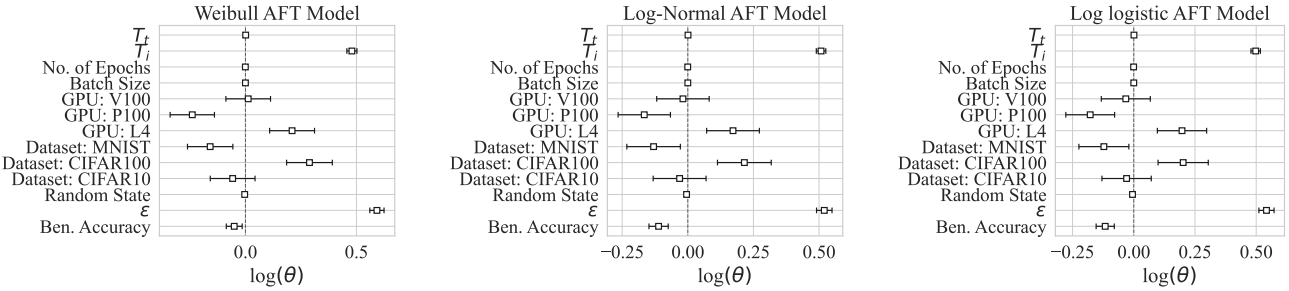

Fig. 7. Coefficients of the Covariates for the Weibull, Log-Normal, and Log-Logistic AFT models. Here, "Random State" is used as a control variable that should be (and is) close to 0. A positive value indicates that a covariate increases the survival time and a negative value indicates that a covariate decreases the survival time. The symbols T_i and T_t refer to training and inference time for all samples, while *Ben. Accuracy* refers to the benign accuracy (accuracy on the un-altered samples), and ε is the noise distance.

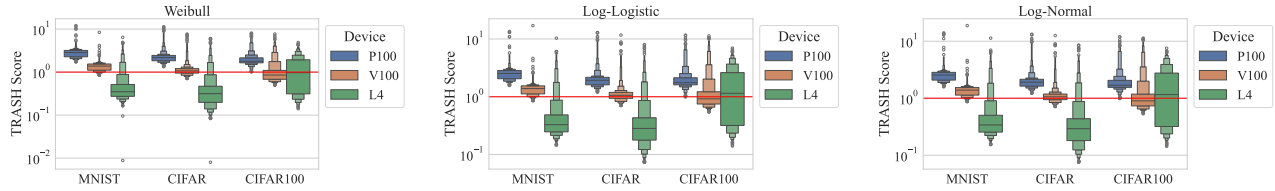

Fig. 8. The *training rate and survival heuristic* score (TRASH score) that depicts the per-sample ratio of training time to attack time for each of the tested AFT models. If this value is greater than one (the red line), then the model can be discarded as ineffective. The x-axis shows each dataset, the y-axis shows the TRASH score and each GPU model is given its own colour. Outlier scores are depicted with a white dot.

techniques. The hyper-parameter search was restricted to a single evasion attack (see Eq. 3) in the interest of time and budget, though this would generalise to evasion, extraction, inversion, or poisoning attacks [9], [10], [18], [51].

VIII. Conclusions

In this work, we presented an autonomic decision-support framework that applies survival analysis to cloud-native ML, enabling rapid and cost-efficient evaluation of model parameter choices in the presence of adversarial noise. Using this technique, adversarial robustness is not substantially affected by reduced run-times from newer and/or larger hardware; the specific choice of attack parameters matters far more than model training time.

The experiments conducted confirm that this methodology is both sound and cost-effective. While 88% of the cloud budget went to GPU rentals, the data show how cutting that cost by 75% and using less power-intensive hardware designed for 8-bit image processing would be *more* effective than using the 32-bit data-centre GPUs (V100 and P100) for typical image classification tasks. Using Kepler proved to be cost-effective, as real-time monitoring only used a fraction of the budget (6%). AFT models fit with concordance > 0.83 on held-out data confirm that survival analysis is a reliable robustness estimator. Attack cost consistently undercut training cost across all hardware and datasets, and AFT coefficients identify inference latency (T_i) — not training time or epoch count — as the dominant robustness predictor.

Furthermore, the coefficients again confirm an inverse relationship between test-set (benign) accuracy and adversarial robustness (*i.e.*, survival time). Finally, even when we account for the reduced GPU bandwidth (see Table I), the L4 model is demonstrably superior in terms of robustness and cost-effectiveness (see Figure 8), despite being advertised as "inference only" by the manufacturer. Together, these findings validate the framework as a quantitative decision-support component for self-adaptive cloud-native deployments: a MAPE-K control loop can confidently select cost-optimal hardware configurations without sacrificing adversarial robustness.

Future work should extend the threat model to black-box, poisoning, and membership inference attacks, testing whether the L4's hardware advantage persists under a broader adversarial surface relevant to production autonomic deployments. The MAPE-K integration should also be validated in live autonomic systems with dynamic workloads, where the Execute phase must adapt hardware configurations at runtime without service interruption. Finally, scaling the AFT-based decision-support to distributed and federated environments would generalise the framework to the multi-node architectures common in cloud-native self-organising systems.

References

[1] PV Srinivas Acharyulu and P Seetharamaiah. A framework for safety automation of safety-critical systems operations. *Safety Science*, 77:133–142, 2015.

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international*

conference on knowledge discovery & data mining, pages 2623–2631, 2019.

[3] Md Zahangir Alom, Tarek M Taha, Christopher Yakopcic, Stefan Westberg, Paheding Sidike, Mst Shamima Nasrin, Brian C Van Esesn, Abdul A S Awwal, and Vijayan K Asari. The history began from alexnet: A comprehensive survey on deep learning approaches. *arXiv preprint arXiv:1803.01164*, 2018.

[4] Marcelo Amaral, Huamin Chen, Tatsuhiro Chiba, Rina Nakazawa, Sunyanan Choochotkaew, Eun Kyung Lee, and Tamar Eilam. Kepler: A framework to calculate the energy consumption of containerized applications. In *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, pages 69–71, 2023.

[5] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3971–3988, 2022.

[6] Peter C Austin, Frank E Harrell Jr, and David van Klaveren. Graphical calibration curves and the integrated calibration index (ICI) for survival models. *Statistics in Medicine*, 39(21):2714–2742, 2020.

[7] C Warren Axelrod. Applying lessons from safety-critical systems to security-critical software. In *2011 IEEE Long Island Systems, Applications and Technology Conference*, pages 1–6. IEEE, 2011.

[8] Alexandre Bailly, Corentin Blanc, Élie Francis, Thierry Guillotin, Fadi Jamal, Béchara Wakim, and Pascal Roy. Effects of dataset size and interactions on the prediction performance of logistic regression and deep learning models. *Computer Methods and Programs in Biomedicine*, 213:106504, 2022.

[9] Battista Biggio, Igino Corona, Davide Maiorca, Blaine Nelson, Nedim Šrndić, Pavel Laskov, Giorgio Giacinto, and Fabio Roli. Evasion attacks against machine learning at test time. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2013, Prague, Czech Republic, September 23-27, 2013, Proceedings, Part III 13*, pages 387–402. Springer, 2013.

[10] Battista Biggio, Blaine Nelson, and Pavel Laskov. Poisoning Attacks against Support Vector Machines. *arXiv:1206.6389 [cs, stat]*, March 2013.

[11] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Prasoon Goyal, Lawrence D Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, et al. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.

[12] Mike J Bradburn, Taane G Clark, Sharon B Love, and Douglas Graham Altman. Survival analysis part II: multivariate data analysis–an introduction to concepts and methods. *British journal of cancer*, 89(3):431–436, 2003.

[13] Tom B Brown, Dandelion Mané, Aurko Roy, Martín Abadi, and Justin Gilmer. Adversarial patch. *arXiv:1712.09665*, 2017.

[14] N. Carlini and D. Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 39–57, 2017.

[15] Anirban Chakraborty, Manaar Alam, Vishal Dey, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. Adversarial attacks and defences: A survey. *arXiv:1810.00069*, 2018.

[16] Anirban Chakraborty, Manaar Alam, Vishal Dey, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. Adversarial attacks and defences: A survey. *arXiv:1810.00069 [cs, stat]*, 2018.

[17] Jianbo Chen, Michael I Jordan, and Martin J Wainwright. HopSkipJumpAttack: A query-efficient decision-based attack. In *IEEE symposium on security and privacy (SP)*, pages 1277–1294. IEEE, 2020.

[18] Christopher A Choquette-Choo, Florian Tramer, Nicholas Carlini, and Nicolas Papernot. Label-only membership inference attacks. In *International conference on machine learning*, pages 1964–1974. PMLR, 2021.

[19] International Electrotechnical Commission. Iec 61508 safety and functional safety, 2010.

[20] Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *International conference on machine learning*, pages 2206–2216. PMLR, 2020.

[21] Cameron Davidson-Pilon. lifelines: survival analysis in python. *Journal of Open Source Software*, 4(40):1317, 2019.

[22] Miyuru Dayarathna, Yonggang Wen, and Rui Fan. Data center energy consumption modeling: A survey. *IEEE Communications surveys & tutorials*, 18(1):732–794, 2015.

[23] Li Deng. The MNIST database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012.

[24] Radosvet Desislavov, Fernando Martínez-Plumed, and José Hernández-Orallo. Compute and energy consumption trends in deep learning inference. *arXiv:2109.05472*, 2021.

[25] Elvis Dohmatob. Generalized No Free Lunch Theorem for Adversarial Robustness. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *PMLR*, 2019.

[26] dvc.org. Dvc- data version control. Github, 2023.

[27] Bradley J Erickson, Panagiotis Korfiatis, Zeynettin Akkus, and Timothy L Kline. Machine learning for medical imaging. *Radiographics*, 37(2):505–515, 2017.

[28] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv:1412.6572*, 2014.

[29] Google. Gpu pricing. compute engine: Virtual machines (vms). google cloud.

[30] Google. Gpu pricing. compute engine: Virtual machines (vms). google cloud.

[31] Wilhelm Hasselbring and Guido Steinacker. Microservice architectures for scalability, agility and reliability in e-commerce. In *2017 IEEE International Conference on Software Architecture Workshops (ICSAW)*, pages 243–246. IEEE, 2017.

[32] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[33] Kaggle. Kaggle datasets, 2026. Accessed: 2026-04-05.

[34] Parves Kamal. A study on the security of password hashing based on gpu based, password cracking using high-performance cloud computing. Master's thesis, St. Cloud State. St. Cloud, Minnesota, USA., 2017.

[35] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *Computer*, 36(1):41–50, 2003.

[36] David G Kleinbaum and Mitchel Klein. *Survival analysis a self-learning text*. Springer, New York, NY, USA, 2012.

[37] Shashank Kotyan and Danilo Vasconcellos Vargas. Adversarial robustness assessment. *PloS one*, 17(4):e0265723, 2022.

[38] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. Technical report, Toronto, ON, Canada, 2009.

[39] Kubernetes. Kubernetes–an open source system for managing containerized applications. Github, June 2019.

[40] John M Lachin. Introduction to sample size determination and power analysis for clinical trials. *Controlled clinical trials*, 2(2):93–113, 1981.

[41] Gaëtan Leurent and Thomas Peyrin. SHA-1 is a shambles: First Chosen-Prefix collision on SHA-1 and application to the PGP web of trust. In *29th USENIX security symposium (USENIX security 20)*, pages 1839–1856, 2020.

[42] Zheng Li and Yang Zhang. Membership leakage in label-only exposures. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 880–895, 2021.

[43] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv:1706.06083*, 2017.

[44] Apoorv Maheshwari, Navindran Davendralingam, and Daniel A DeLaurentis. A comparative study of machine learning techniques for aviation applications. In *2018 Aviation Technology, Integration, and Operations Conference*, page 3980, 2018.

[45] International Standards Organization. ISO 26262-1:2011, road vehicles — functional safety. https://www.iso.org/standard/43464.html, 2018.

[46] Domingo Mery, Erick Svec, Marco Arias, Vladimir Riffo, Jose M Saavedra, and Sandipan Banerjee. Modern computer vision techniques for x-ray testing in baggage inspection. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 47(4):682–692, 2016.

[47] Charles Meyers, Tommy Löfstedt, and Erik Elmroth. Safety-critical computer vision: An empirical survey of adversarial evasion attacks and defenses on computer vision systems. *Artificial Intelligence Review*, 2023.

[48] Charles Meyers, Mohammad Reza, Tommy Löfstedt, and Erik Elmroth. A systematic approach to robustness modelling. In *Accepted for International Conference on Machine Learning and Cybernetics*, 2023.

[49] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. Deepfool: a simple and accurate method to fool deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2574–2582, 2016.

[50] Maria Irina Nicolae, Mathieu Sinn, Minh Ngoc Tran, Beat Buesser, Ambrish Rawat, Martin Wistuba, Valentina Zantedeschi, Nathalie Baracaldo, Bryant Chen, Heiko Ludwig, Ian Molloy, and Ben Edwards. Adversarial robustness toolbox v1.2.0. *CoRR*, 1807.01069, 2018.

[51] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Knockoff nets: Stealing functionality of black-box models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4954–4963, 2019.

[52] Yoshihiko Ozaki, Yuki Tanigaki, Shuhei Watanabe, and Masaki Onishi. Multiobjective tree-structured parzen estimator for computationally expensive optimization problems. In *Proceedings of the 2020 genetic and evolutionary computation conference*, pages 533–541, 2020.

[53] Matt O'Brien, Hannah Fingerhut, and The Associated Press. A.i. tools fueled a 34% spike in microsoft's water consumption, and one city with its data centers is concerned about the effect on residential supply, Sep 2023.

[54] D Panchal, P Verma, I Baran, D Musgrove, and D Lu. Reusable mlops: Reusable deployment, reusable infrastructure and hot-swappable machine learning models and services. *arXiv preprint arXiv:2403.00787*, 2024.

[55] Aniruddha Saha, Akshayvarun Subramanya, and Hamed Pirsiavash. Hidden trigger backdoor attacks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 11957–11965, 2020.

[56] Mohammad Sajid and Zahid Raza. Cloud computing: Issues & challenges. In *International conference on cloud, big data and trust*, volume 20, pages 13–15, 2013.

[57] Claudia Schmoor, Willi Sauerbrei, and Martin Schumacher. Sample size considerations for the evaluation of prognostic factors in survival analysis. *Statistics in medicine*, 19(4):441–452, 2000.

[58] Mohammad Reza Saleh Sedghpour and Paul Townend. Service mesh and ebpf-powered microservices: A survey and future directions. In *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, pages 176–184, 2022.

[59] Shai Shalev-Shwartz and Shai Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge university press, Cambridge, UK, 2014.

[60] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[61] Neelam Singh, Yasir Hamid, Sapna Juneja, Gautam Srivastava, Gaurav Dhiman, Thippa Reddy Gadekallu, and Mohd Asif Shah. Load balancing and service discovery using docker swarm for microservice based big data applications. *Journal of Cloud Computing*, 12(1):4, 2023.

[62] Chen Sun, Abhinav Shrivastava, Saurabh Singh, and Abhinav Gupta. Revisiting unreasonable effectiveness of data in deep learning era. In *Proceedings of the IEEE international conference on computer vision*, pages 843–852, 2017.

[63] Martin Svedin, Steven WD Chien, Gibson Chikafa, Niclas Jansson, and Artur Podobas. Benchmarking the nvidia gpu lineage: From early k80 to modern a100 with asynchronous memory transfers. In *Proceedings of the 11th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies*, pages 1–6, 2021.

[64] Jimin Tan, Jianan Yang, Sai Wu, Gang Chen, and Jake Zhao. A critical look at the current train/test split in machine learning. *arXiv preprint arXiv:2106.04525*, 2021.

[65] Guido Vittorio Travaini, Federico Pacchioni, Silvia Bellumore, Marta Bosia, and Francesco De Micco. Machine learning and criminal justice: A systematic review of advanced methodology for recidivism risk prediction. *International journal of environmental research and public health*, 19(17):10594, 2022.

[66] Vladimir N Vapnik. An overview of statistical learning theory. *IEEE transactions on neural networks*, 10(5):988–999, 1999.

[67] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, et al. An empirical study of mamba-based language models. *arXiv preprint arXiv:2406.07887*, 2024.

[68] Shuhei Watanabe. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. *arXiv preprint arXiv:2304.11127*, 2023.

[69] Rengan Xu, Frank Han, and Quy Ta. Deep learning at scale on nvidia v100 accelerators. In *2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, pages 23–32. IEEE, 2018.

[70] Weilin Xu, David Evans, and Yanjun Qi. Feature squeezing: Detecting adversarial examples in deep neural networks. *arXiv:1704.01155*, 2017.

[71] Omry Yadan. Hydra - a framework for elegantly configuring complex applications. Github, 2019.

[72] Ruiting Zhou, Jinlong Pang, Qin Zhang, Chuan Wu, Lei Jiao, Yi Zhong, and Zongpeng Li. Online scheduling algorithm for heterogeneous distributed machine learning jobs. *IEEE Transactions on Cloud Computing*, 2022.

[73] Eckart Zitzler, Joshua Knowles, and Lothar Thiele. Quality assessment of pareto set approximations. *Multiobjective optimization: Interactive and evolutionary approaches*, pages 373–404, 2008.